

# cppbuild

mccakit · 2026-06-03

## Introduction

cppbuild is a C/C++ build system with Umka frontend and C++ backend.

### Why a new build system?

With the entry of modules with C++20, compilation model of C++ changed. Modules, unlike translation units depend on compilation order. Packaging them requires the user to build interfaces from source or ensuring all bmi's are built with the same compiler, flags etc.

This is hard to do, and harder to do well, in current C++ build systems. The reasons include backwards compatibility, the urge to support everything, and hand-holding users who don't understand compiler flags. The result is bloat.

CMake is 300K lines, Meson 75K — yet Ninja, the build system actually doing the work, is 20K. Both are starved for manpower, and every issue they close is replaced by several more.

Documentation of these codebases is also lacking, there isn't any document targeting contributors beyond guidelines, and user documentation is often outdated and incomplete.

There are also several other issues, build systems modify and interpret your flags, they play package manager and have api's to call other build systems. They have terrible scripting DSL.

The building system's job is to execute a build and package/copy the result, nothing more.

### Design goals

- Never modify flags; pass them through unchanged.
- Never fetch dependencies.
- Ship as a native binary with minimal dependencies.
- Limited Scope. C/C++; Linux, Windows, MacOS; Clang, GCC and MSVC recent releases. Ninja backend only.
- Staticly typed, embedded scripting
- Detailed documentation covering users and contributors needs
- Comprehensive testing/benchmark suite.
- Prioritize incremental build performance over full build time
- Build it with C++ modules. Port existing libraries to modules when required.